KARPAGAM
COLLEGE OF ENGINEERING
Rediscover | Refine | Redefine
Accredited by NAAC with 'A+' grade
Autonomous | Affiliated to Anna University
(An ISO 9001:2015 and ISO 14001:2015 Certified Institution)

ACADEMIC YEAR: 2026-27

ASSIGNMENT FRONT SHEET

Name of the Student:	**DHARSHANA T**
Register Number:	**717825P211**
Course Code & Title:	**23CSR306 JAVA PROGRAMMING**
Year / Dept. / Sem / Section:	**II / CSE/ III /B**
Assignment:	**II – OOP Mini Project (any one of 35)**
Specification No. & Title:	**4 - Hotel Reservation System**
GitHub Repository Link:	**https://github.com/Dharshu-23/HotelReservationSystem.git**
Date of Issue:	
Date of Submission:	

Marks Details

CO / K Level	Maximum Marks	Marks Obtained
CO4 / K3	50	
CO5 / K3	50	
Total	**100**	

Course In-charge

KARPAGAM COLLEGE OF ENGINEERING

Rubrics for Assignment Evaluation

Assignment Number	:	II	**Date**	:	
Course Code & Title	:	23CSR306 Java Programming	**Specification No. & Title**	:	4 - Hotel Reservation System
Register Number	:	717825P211	**Name of the Student**	:	Dharshana T
Name of the Evaluator	:		**GitHub Link**	:	https://github.com/Dharshu-23/HotelReservationSystem.git

Marks for each criterion = Level (1 – 5) × Weightage. Total: 100 (CO4: 50 + CO5: 50).

Criterion	CO	Wt.	Exemplary (5)	Excellent (4)	Adequate (3)	Average (2)	Inadequate (1)	Max	Marks
Program Correctness & Business Rules	CO4	**4**	All menu options work; every business rule enforced correctly.	All options work; minor gaps in one rule.	Main flow works; some rules missing.	Partially working; several rules not handled.	Does not run or most features missing.	20	
Input Validation & Exception Handling	CO4	**3**	All inputs validated; custom exceptions; program never crashes.	Most inputs validated with try-catch.	Basic validation; some invalid inputs not handled.	Little validation; crashes on bad input.	No validation or exception handling.	15	
Menu-driven Interface & Expected Outputs	CO4	**3**	Clear menu; all expected outputs (bills/receipts/status) shown neatly.	Clear menu; most expected outputs shown.	Menu works; outputs basic or incomplete.	Menu unclear; few outputs shown.	No proper menu or outputs.	15	
OOP Design (Encapsulation, Inheritance, Aggregation, Polymorphism)	CO5	**4**	All four concepts applied meaningfully; all required classes present.	Concepts applied well; minor design gaps.	Classes present; concepts used in a basic way.	Weak design; some required classes missing.	No OOP design; mostly procedural.	20	
Code Quality & Package Structure	CO5	**2**	Clean packages, naming, indentation and comments.	Good structure with few lapses.	Acceptable structure; average readability.	Poor naming / structure.	Single file; unreadable code.	10	
GitHub Repository & README	CO5	**1**	Complete code with clear README (problem, run steps, name, roll no.).	Complete code; README missing minor details.	Code present; README incomplete.	Partial code; no README.	No repository.	5	
Viva / Explanation	CO5	**3**	Explains design, rules and code clearly; answers all questions.	Explains well; answers most questions.	Explains main logic; answers some questions.	Limited explanation of own code.	Unable to explain the code.	15	
							CO4 Total (Criteria 1 – 3)	**50**	
							CO5 Total (Criteria 4 – 7)	**50**	
							Total Marks	**100**	

Signature of the Evaluator with date

1. Assignment Question

Specification 4: Hotel Reservation System

Problem Statement: Design and implement a Java console application for a Hotel Reservation System to manage rooms, customers, bookings, and billing.

Class Requirements: Room, Customer, PremiumCustomer, Booking, Payment, Bill, Hotel

Business Rules:

1. Bookings can only be confirmed if rooms are available.

2. Room availability updates immediately after booking or cancellation.

3. Premium customers may receive discounts or loyalty points.

4. Billing includes taxes and optional discounts.

5. Each booking links directly to a room and customer.

Menu Options: Add Room / Add Customer / Make Reservation / Cancel Reservation / Display Rooms / Exit

Expected Output:

• Show reservation confirmation with details.

• Display updated room availability.

• Show bills with discounts and taxes.

2. Java Source Code

HotelReservationSystem.java:

```
import java.sql.*;
import java.util.Scanner;
public class HotelReservationSystem {
    private static final Scanner sc = new Scanner(System.in);
    private static final double TAX_RATE = 0.05;
    static class Room {
        int roomNo;
        String type;
```

```
    double price;
    boolean available;
    Room(int roomNo, String type, double price,
         boolean available) {
        this.roomNo = roomNo;
        this.type = type;
        this.price = price;
        this.available = available;
    }
}
static class Customer {
    int id;
    String name;
    String phone;
    String type;
    int points;
    Customer(int id, String name, String phone,
            String type, int points) {
        this.id = id;
        this.name = name;
        this.phone = phone;
        this.type = type;
        this.points = points;
    }
    double getDiscount() {
        return type.equalsIgnoreCase("Premium") ? 10.0 : 0.0;
    }
```

```
    }

    static class Bill {
        double roomCharge;
        double discount;
        double tax;
        double total;
        Bill(double price, int nights, double discountPercent) {
            roomCharge = price * nights;
            discount = roomCharge * discountPercent / 100.0;
            tax = (roomCharge - discount) * TAX_RATE;
            total = roomCharge - discount + tax;
        }
        void display() {
            System.out.println("\n========== HOTEL BILL ==========");
            System.out.printf("Room charges : Rs.%.2f%n", roomCharge);
            System.out.printf("Discount     : Rs.%.2f%n", discount);
            System.out.printf("Tax (5%%)     : Rs.%.2f%n", tax);
            System.out.printf("Total        : Rs.%.2f%n", total);
            System.out.println("================================");
        }
    }
    static void addRoom() {
        try (Connection con = DBConnection.getConnection()) {
            System.out.print("Room number: ");
            int no = Integer.parseInt(sc.nextLine());
            System.out.print("Room type: ");
            String type = sc.nextLine().trim();
```

```
        System.out.print("Price per night: ");
        double price = Double.parseDouble(sc.nextLine());
        if (no <= 0 || type.isEmpty() || price <= 0) {
            System.out.println("Enter valid room details.");
            return;
        }
        String sql = "INSERT INTO rooms "
                + "(room_no, room_type, price, available) "
                + "VALUES (?, ?, ?, TRUE)";

        try (PreparedStatement ps = con.prepareStatement(sql)) {
            ps.setInt(1, no);
            ps.setString(2, type);
            ps.setDouble(3, price);
            ps.executeUpdate();
        }
        System.out.println("Room added successfully.");
    } catch (SQLException e) {
        System.out.println("Database error: " + e.getMessage());
    } catch (NumberFormatException e) {
        System.out.println("Enter valid numeric values.");
    }
}
static void addCustomer() {
    try (Connection con = DBConnection.getConnection()) {
        System.out.print("Customer ID: ");
        int id = Integer.parseInt(sc.nextLine());
```

```
        System.out.print("Customer name: ");
        String name = sc.nextLine().trim();
        System.out.print("Phone number: ");
        String phone = sc.nextLine().trim();
        System.out.print("Premium customer? (yes/no): ");
        String answer = sc.nextLine();
        if (id <= 0 || name.isEmpty()) {
            System.out.println("Enter valid customer details.");
            return;
        }
        String type = answer.equalsIgnoreCase("yes")
                ? "Premium" : "Regular";

        String sql = "INSERT INTO customers "
                + "(customer_id, name, phone, customer_type) "
                + "VALUES (?, ?, ?, ?)";

        try (PreparedStatement ps = con.prepareStatement(sql)) {
            ps.setInt(1, id);
            ps.setString(2, name);
            ps.setString(3, phone);
            ps.setString(4, type);
            ps.executeUpdate();
        }
        System.out.println(type + " customer added successfully.");
    } catch (SQLException e) {
        System.out.println("Database error: " + e.getMessage());
```

```
        } catch (NumberFormatException e) {
            System.out.println("Customer ID must be numeric.");
        }
    }
    static Room findAvailableRoom(Connection con, int roomNo)
            throws SQLException {
        String sql = "SELECT room_no, room_type, price, available "+ "FROM rooms WHERE room_no = ? FOR
UPDATE";
        try (PreparedStatement ps = con.prepareStatement(sql)) {
            ps.setInt(1, roomNo);
            try (ResultSet rs = ps.executeQuery()) {
                if (rs.next()) {
                    return new Room(
                        rs.getInt("room_no"),
                        rs.getString("room_type"),
                        rs.getDouble("price"),
                        rs.getBoolean("available")
                    );
                }
            }
        }
        return null;
    }
    static Customer findCustomer(Connection con, int id)
            throws SQLException {
        String sql = "SELECT * FROM customers WHERE customer_id = ?";
        try (PreparedStatement ps = con.prepareStatement(sql)) {
```

```
            ps.setInt(1, id);
            try (ResultSet rs = ps.executeQuery()) {
                if (rs.next()) {
                    return new Customer(
                        rs.getInt("customer_id"),
                        rs.getString("name"),
                        rs.getString("phone"),
                        rs.getString("customer_type"),
                        rs.getInt("loyalty_points")
                    );
                }
            }
        }
        return null;
    }
    static void makeReservation() {
        try (Connection con = DBConnection.getConnection()) {
            con.setAutoCommit(false);

            try {
                System.out.print("Room number: ");
                int roomNo = Integer.parseInt(sc.nextLine());
                Room room = findAvailableRoom(con, roomNo);
                if (room == null || !room.available) {
                    System.out.println("Room not found or unavailable.");
                    con.rollback();
                    return;
```

```
        }
        System.out.print("Customer ID: ");
        int customerId = Integer.parseInt(sc.nextLine());
        Customer customer = findCustomer(con, customerId);
        if (customer == null) {
            System.out.println("Customer not found.");
            con.rollback();
            return;
        }
        System.out.print("Number of nights: ");
        int nights = Integer.parseInt(sc.nextLine());
        if (nights <= 0) {
            System.out.println("Nights must be positive.");
            con.rollback();
            return;
        }
        String updateRoom =
            "UPDATE rooms SET available = FALSE "
          + "WHERE room_no = ? AND available = TRUE";
        try (PreparedStatement ps =
                 con.prepareStatement(updateRoom)) {
            ps.setInt(1, roomNo);
            if (ps.executeUpdate() != 1) {
                con.rollback();
                System.out.println("Room is no longer available.");
                return;
            }
```

```
        }
        String insertBooking =
            "INSERT INTO bookings (room_no, customer_id, nights, status) "
          + "VALUES (?, ?, ?, 'CONFIRMED')";

        int bookingId;
        try (PreparedStatement ps = con.prepareStatement(
                insertBooking, Statement.RETURN_GENERATED_KEYS)) {
            ps.setInt(1, roomNo);
            ps.setInt(2, customerId);
            ps.setInt(3, nights);
            ps.executeUpdate();
            try (ResultSet keys = ps.getGeneratedKeys()) {
                if (!keys.next()) {
                    throw new SQLException("Booking ID not generated.");
                }
                bookingId = keys.getInt(1);
            }
        }
        Bill bill = new Bill(
            room.price, nights, customer.getDiscount()
        );
        String insertPayment =
            "INSERT INTO payments "
          + "(booking_id, room_charge, discount, tax, total) "
          + "VALUES (?, ?, ?, ?, ?)";
        try (PreparedStatement ps =
```

```
            con.prepareStatement(insertPayment)) {
        ps.setInt(1, bookingId);
        ps.setDouble(2, bill.roomCharge);
        ps.setDouble(3, bill.discount);
        ps.setDouble(4, bill.tax);
        ps.setDouble(5, bill.total);
        ps.executeUpdate();
    }
    int earnedPoints = (int) (bill.total / 100);
    if (customer.type.equalsIgnoreCase("Premium")) {
        String updatePoints =
            "UPDATE customers SET loyalty_points = "
          + "loyalty_points + ? WHERE customer_id = ?";
        try (PreparedStatement ps =
                con.prepareStatement(updatePoints)) {
            ps.setInt(1, earnedPoints);
            ps.setInt(2, customerId);
            ps.executeUpdate();
        }
    }
    con.commit();
    System.out.println("\nReservation confirmed!");
    System.out.println("Booking ID: " + bookingId);
    System.out.println("Customer: " + customer.name);
    System.out.println("Room: " + room.roomNo + " (" + room.type + ")");
    System.out.println("Nights: " + nights);
    bill.display();
```

```
            if (customer.type.equalsIgnoreCase("Premium")) {
                System.out.println("Loyalty points earned: " + earnedPoints);
            }
            System.out.println("Room availability updated: Booked.");
        } catch (Exception e) {
            con.rollback();
            System.out.println("Reservation failed: " + e.getMessage());
        } finally {
            con.setAutoCommit(true);
        }
    } catch (SQLException e) {
        System.out.println("Database error: " + e.getMessage());
    }
}
static void cancelReservation() {
    try (Connection con = DBConnection.getConnection()) {
        con.setAutoCommit(false);
        try {
            System.out.print("Booking ID: ");
            int id = Integer.parseInt(sc.nextLine());
            int roomNo;
            String sql = "SELECT room_no FROM bookings "
                    + "WHERE booking_id = ? AND status = 'CONFIRMED' "
                    + "FOR UPDATE";
            try (PreparedStatement ps = con.prepareStatement(sql)) {
                ps.setInt(1, id);
                try (ResultSet rs = ps.executeQuery()) {
```

```
            if (!rs.next()) {
                System.out.println(
                    "Active booking not found."
                );
                con.rollback();
                return;
            }
            roomNo = rs.getInt("room_no");
        }
    }
    try (PreparedStatement ps = con.prepareStatement(
            "UPDATE bookings SET status = 'CANCELLED' "
          + "WHERE booking_id = ?")) {
        ps.setInt(1, id);
        ps.executeUpdate();
    }
    try (PreparedStatement ps = con.prepareStatement(
            "UPDATE rooms SET available = TRUE WHERE room_no = ?")) {
        ps.setInt(1, roomNo);
        ps.executeUpdate();
    }
    con.commit();
    System.out.println("Reservation cancelled successfully.");
    System.out.println("Room " + roomNo + " is now available.");
} catch (Exception e) {
    con.rollback();
    System.out.println("Cancellation failed: " + e.getMessage());
```

```
            } finally {
                con.setAutoCommit(true);
            }
        } catch (SQLException e) {
            System.out.println("Database error: " + e.getMessage());
        }
    }
    static void displayRooms() {
        String sql = "SELECT * FROM rooms ORDER BY room_no";
        try (Connection con = DBConnection.getConnection();
             Statement st = con.createStatement();
             ResultSet rs = st.executeQuery(sql)) {
            System.out.println("\n========== ROOMS ==========");
            while (rs.next()) {
                System.out.printf(
                    "Room: %d | Type: %s | Price: Rs.%.2f | Status: %s%n",
                    rs.getInt("room_no"),
                    rs.getString("room_type"),
                    rs.getDouble("price"),
                    rs.getBoolean("available") ? "Available" : "Booked"
                );
            }
        } catch (SQLException e) {
            System.out.println("Database error: " + e.getMessage());
        }
    }
    static void displayCustomers() {
```

```
        String sql = "SELECT * FROM customers ORDER BY customer_id";
        try (Connection con = DBConnection.getConnection();
             Statement st = con.createStatement();
             ResultSet rs = st.executeQuery(sql)) {
            while (rs.next()) {
                System.out.printf(
                    "ID: %d | Name: %s | Type: %s | Points: %d%n",
                    rs.getInt("customer_id"),
                    rs.getString("name"),
                    rs.getString("customer_type"),
                    rs.getInt("loyalty_points")
                );
            }
        } catch (SQLException e) {
            System.out.println("Database error: " + e.getMessage());
        }
    }
    static void displayBookings() {
        String sql =
            "SELECT b.booking_id, b.room_no, c.name, b.nights, b.status "
          + "FROM bookings b JOIN customers c "
          + "ON b.customer_id = c.customer_id ORDER BY b.booking_id";
        try (Connection con = DBConnection.getConnection();
             Statement st = con.createStatement();
             ResultSet rs = st.executeQuery(sql)) {
            while (rs.next()) {
                System.out.printf(
```

```
                "Booking: %d | Customer: %s | Room: %d "
              + "| Nights: %d | Status: %s%n",
                rs.getInt("booking_id"),
                rs.getString("name"),
                rs.getInt("room_no"),
                rs.getInt("nights"),
                rs.getString("status")
            );
        }
    } catch (SQLException e) {
        System.out.println("Database error: " + e.getMessage());
    }
}
static void displayBills() {
    String sql =
        "SELECT p.payment_id, p.booking_id, p.room_charge, "
      + "p.discount, p.tax, p.total "
      + "FROM payments p ORDER BY p.payment_id";
    try (Connection con = DBConnection.getConnection();
         Statement st = con.createStatement();
         ResultSet rs = st.executeQuery(sql)) {
        while (rs.next()) {
            System.out.printf(
                "Payment: %d | Booking: %d | Charge: Rs.%.2f "
              + "| Discount: Rs.%.2f | Tax: Rs.%.2f | Total: Rs.%.2f%n",
                rs.getInt("payment_id"),
                rs.getInt("booking_id"),
```

```
                rs.getDouble("room_charge"),
                rs.getDouble("discount"),
                rs.getDouble("tax"),
                rs.getDouble("total")
            );
        }
    } catch (SQLException e) {
        System.out.println("Database error: " + e.getMessage());
    }
}
public static void main(String[] args) {
    try (Connection con = DBConnection.getConnection()) {
        System.out.println("MySQL connected successfully!");
    } catch (SQLException e) {
        System.out.println("Cannot connect to MySQL: " + e.getMessage());
        System.out.println("Check the server, database and password.");
        return;
    }
    while (true) {
        System.out.println("\n===== HOTEL RESERVATION SYSTEM =====");
        System.out.println("1. Add Room");
        System.out.println("2. Add Customer");
        System.out.println("3. Make Reservation");
        System.out.println("4. Cancel Reservation");
        System.out.println("5. Display Rooms");
        System.out.println("6. Display Customers");
        System.out.println("7. Display Bookings");
```

```
            System.out.println("8. Display Bills");
            System.out.println("9. Exit");
            System.out.print("Choice: ");
            String input = sc.nextLine();
            switch (input) {
                case "1": addRoom(); break;
                case "2": addCustomer(); break;
                case "3": makeReservation(); break;
                case "4": cancelReservation(); break;
                case "5": displayRooms(); break;
                case "6": displayCustomers(); break;
                case "7": displayBookings(); break;
                case "8": displayBills(); break;
                case "9":
                    System.out.println("Thank you!");
                    return;
                default:
                    System.out.println("Invalid choice.");
            }
        }
    }
}
```

DBConnection.java:

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;
```

```
public class DBConnection {

    private static final String URL =

        "jdbc:mysql://localhost:3306/hotel_db";

    private static final String USER = "root";

    private static final String PASSWORD =

        "Dharshana";

    public static Connection getConnection()

            throws SQLException {

        return DriverManager.getConnection(

            URL, USER, PASSWORD

        );

    }

}
```

3. Execution Output Screenshots

```
MySQL connected successfully!

===== HOTEL RESERVATION SYSTEM =====
1. Add Room
2. Add Customer
3. Make Reservation
4. Cancel Reservation
5. Display Rooms
6. Display Customers
7. Display Bookings
8. Display Bills
9. Exit
Choice: 1
Room number: 201
Room type: luxure
Price per night: 250000
Room added successfully.
```

```
===== HOTEL RESERVATION SYSTEM =====
1. Add Room
2. Add Customer
3. Make Reservation
4. Cancel Reservation
5. Display Rooms
6. Display Customers
7. Display Bookings
8. Display Bills
9. Exit
Choice: 5

========== ROOMS ==========
Room: 201 | Type: luxure | Price: Rs.250000.00 | Status: Available

===== HOTEL RESERVATION SYSTEM =====
1. Add Room
2. Add Customer
3. Make Reservation
4. Cancel Reservation
5. Display Rooms
6. Display Customers
7. Display Bookings
8. Display Bills
9. Exit
Choice: 2
Customer ID: 101
Customer name: Dharshana
Phone number: 8072282509
```

```
Premium customer? (yes/no): yes
Premium customer added successfully.

===== HOTEL RESERVATION SYSTEM =====
1. Add Room
2. Add Customer
3. Make Reservation
4. Cancel Reservation
5. Display Rooms
6. Display Customers
7. Display Bookings
8. Display Bills
9. Exit
Choice: 3
Room number: 202
Room not found or unavailable.

===== HOTEL RESERVATION SYSTEM =====
1. Add Room
2. Add Customer
3. Make Reservation
4. Cancel Reservation
5. Display Rooms
6. Display Customers
7. Display Bookings
8. Display Bills
9. Exit
Choice: 3
Room number: 201
```

```
Customer ID: 101
Number of nights: 3

Reservation confirmed!
Booking ID: 1
Customer: Dharshana T
Room: 201 (luxure)
Nights: 3

========== HOTEL BILL ==========
Room charges : Rs.750000.00
Discount     : Rs.75000.00
Tax (5%)     : Rs.33750.00
Total        : Rs.708750.00
================================
Loyalty points earned: 7087
Room availability updated: Booked.

===== HOTEL RESERVATION SYSTEM =====
1. Add Room
2. Add Customer
3. Make Reservation
4. Cancel Reservation
5. Display Rooms
6. Display Customers
7. Display Bookings
8. Display Bills
9. Exit
Choice: 4
```

```
Booking ID: 1
Reservation cancelled successfully.
Room 201 is now available.

===== HOTEL RESERVATION SYSTEM =====
1. Add Room
2. Add Customer
3. Make Reservation
4. Cancel Reservation
5. Display Rooms
6. Display Customers
7. Display Bookings
8. Display Bills
9. Exit
Choice: 5

========== ROOMS ==========
Room: 201 | Type: luxure | Price: Rs.250000.00 | Status: Available

===== HOTEL RESERVATION SYSTEM =====
1. Add Room
2. Add Customer
3. Make Reservation
4. Cancel Reservation
5. Display Rooms
6. Display Customers
7. Display Bookings
8. Display Bills
9. Exit
```

```
Choice: 6
ID: 101 | Name: Dharshana T | Type: Premium | Points: 7087

===== HOTEL RESERVATION SYSTEM =====
1. Add Room
2. Add Customer
3. Make Reservation
4. Cancel Reservation
5. Display Rooms
6. Display Customers
7. Display Bookings
8. Display Bills
9. Exit
Choice: 7
Booking: 1 | Customer: Dharshana T | Room: 201 | Nights: 3 | Status: CANCELLED

===== HOTEL RESERVATION SYSTEM =====
1. Add Room
2. Add Customer
3. Make Reservation
4. Cancel Reservation
5. Display Rooms
6. Display Customers
7. Display Bookings
8. Display Bills
9. Exit
Choice: 8
Payment: 1 | Booking: 1 | Charge: Rs.750000.00 | Discount: Rs.75000.00 | Tax: Rs.33750.00 | Total: Rs.708750.00
```

```
===== HOTEL RESERVATION SYSTEM =====
1. Add Room
2. Add Customer
3. Make Reservation
4. Cancel Reservation
5. Display Rooms
6. Display Customers
7. Display Bookings
8. Display Bills
9. Exit
Choice: 9
Thank you!
PS D:\HotelReservationSystem>
```

4. GitHub Repository Link

https://github.com/Dharshu-23/HotelReservationSystem.git